

How RxInsight Works

Architecture, data flow and design decisions

RxInsight · github.com/Swapnil-byte-798/rxinsight · Swapnil Dwivedi

RxInsight is a small pharmaceutical commercial analytics warehouse: a PostgreSQL star schema, a Python ETL that maintains a Type 2 slowly changing dimension, and four window-function reports over roughly 2.2 million prescription rows.

It assumes you can program but not that you know pharma or dimensional warehousing. Every number was measured on a real run against the loaded database; where something did not work, it says so. All data is synthetic, generated by `etl/generate_data.py` with a fixed seed (42) — no real patient, prescriber or commercial data exists anywhere in this project.

1. The problem

A pharmaceutical brand team sells a drug through field representatives who visit doctors — health care professionals, or HCPs — and detail the product to them. That visit is a *call*. Prescriptions are counted as **TRx** (total, including refills) and **NRx** (new starts only). Reps work *territories*; each HCP carries a *decile*, a 1-to-10 value segment.

A transactional system tells the team how many scripts were written last month. That is almost never the question anyone has. The questions are:

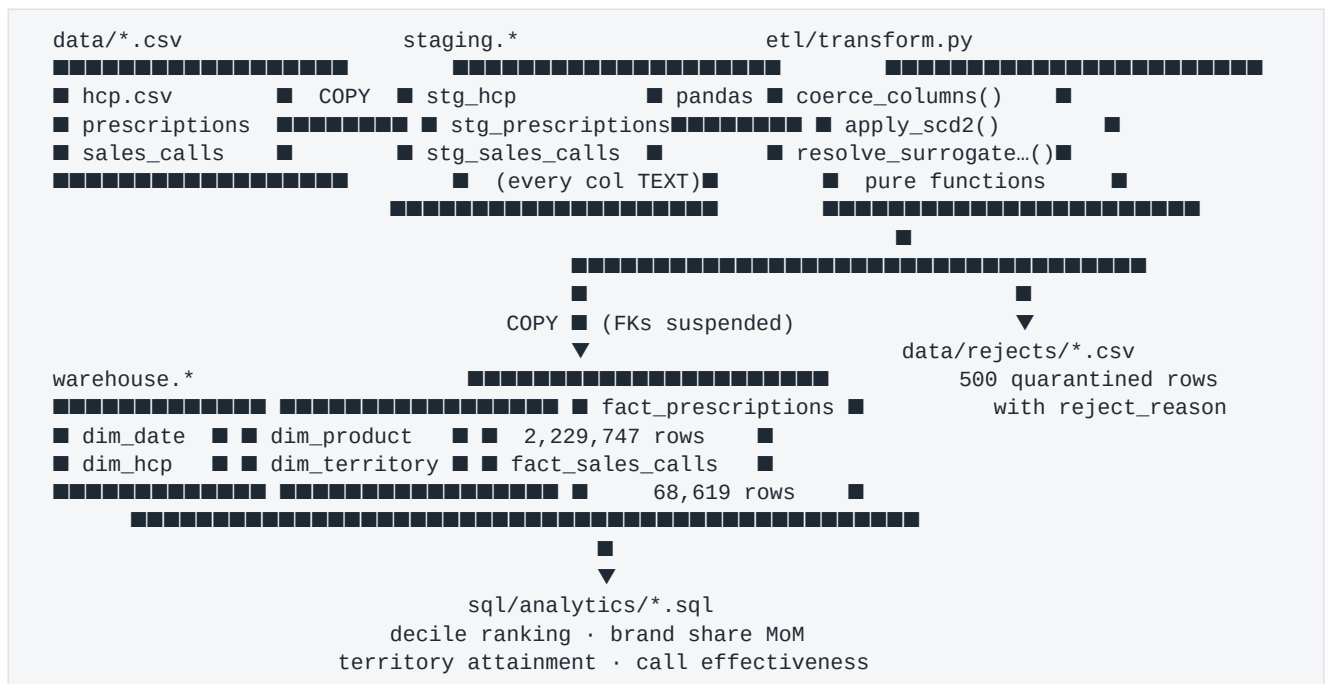
1 **Which prescribers are actually driving our brand?** Call capacity is finite, so the answer has to be a ranked call plan per territory, not a national total.

1 **Which territories are behind plan, and by how much?** Monthly noise is not interesting; the year-to-date hole is.

1 **Do our sales calls move prescriptions**, or is the field force calling on doctors who would have written anyway?

A plain transactional database answers these badly. The answers are aggregates over millions of rows sliced several ways at once, and a normalised OLTP schema makes you join five tables to get there. Worse, **OLTP tables overwrite history**: when a doctor transfers territory, the source system updates one row and moves on. If the warehouse did that, every prescription that doctor wrote last January would retroactively belong to the new territory and the rep who covered the old one would silently lose credit. Question 2 cannot be answered correctly without remembering what was true at the time.

2. The shape of the system



Extract. `etl/generate_data.py` writes three CSVs mimicking what a commercial data vendor hands a brand team: an HCP master, a prescription feed, a sales-call log. It plants a genuine 14-day post-call lift so the analytics have a signal to find, and deliberately corrupts 500 rows so the reject path has something real to catch.

Stage. `etl/load_staging.py` streams each CSV from disk into `staging.*` with `COPY ... FROM STDIN`. Nothing is validated here; staging exists to get bytes into the database fast.

Transform. `etl/transform.py` is pure Python over pandas DataFrames — no connections, no file I/O, no clock. It casts the TEXT columns, folds new HCP versions into the Type 2 dimension, and swaps natural keys for surrogate keys, returning (*clean*, *rejects*) rather than raising on bad data.

Load and analyse. The fact path runs in batches rather than all at once: staging is read 250,000 rows at a time through a server-side cursor, and each batch is typed, key-resolved, `COPY`d and released before the next is fetched, so peak memory is set by the batch size rather than by the table (442 MB measured across the full 2.2M-row load). `etl/pipeline.py` does this with foreign keys temporarily suspended, then reconciles: staged must equal loaded plus rejected, or the run fails. Four `psql`-runnable SQL files in `sql/analytics/` then read the star schema directly, with no application code in between.

3. The star schema

Four **conformed dimensions** are shared by two fact tables at different grains.

Table	Grain	Rows
<code>dim_date</code>	one row per calendar day, keyed YYYYMMDD	730 (2024-01-01 to 2025-12-30)
<code>dim_product</code>	one row per marketed brand	8 (3 own-brand, 5 competitor)
<code>dim_territory</code>	one row per sales territory	50
<code>dim_hcp</code>	one row per <i>version</i> of an HCP	2,038 rows covering 2,000 HCPs
<code>fact_prescriptions</code>	one row per HCP per product per day	2,229,747
<code>fact_sales_calls</code>	one row per rep call on one HCP for one product	68,619

Conformed means both facts point at the same dimension rows with the same keys, so one `WHERE` clause slices both identically — `WHERE d.year_month = '2024-06'` means exactly the same thing to prescriptions and to calls. `call_effectiveness.sql` depends on this directly: it aggregates both facts to HCP-day through `dim_date` and

range-joins them. With purpose-built report tables instead, that join would be a negotiation.

`dim_product.is_competitor` makes share analysis possible: our molecules and the tracked competitor molecules live in one table, so the share denominator is a SUM over the whole market and own-brand reports just add `WHERE is_competitor = FALSE`. `territory_key` is carried on both facts rather than joined through `dim_hcp` — deliberate denormalisation that keeps territory rollups correct after a transfer without walking version history at query time.

4. Slowly Changing Dimension Type 2, worked through

`dim_hcp` tracks two attributes for change: `decile` and `territory_key`. A change to either creates a new row with a new surrogate key; a change to anything else — a name correction, say — does not.

Here are actual rows from the loaded warehouse. Dr. Kavita Sharma (HCP-00242) moves from territory T-035 to T-040 partway through the period. Before the move:

hcp_key	hcp_id	full_name	decile	territory	valid_from	valid_to	is_current
255	HCP-00242	Dr. Kavita Sharma	5	T-035	2024-01-01	9999-12-31	true

After `apply_scd2` folds in the change record with `effective_date = 2024-10-03`:

hcp_key	hcp_id	full_name	decile	territory	valid_from	valid_to	is_current
255	HCP-00242	Dr. Kavita Sharma	5	T-035	2024-01-01	2024-10-02	false
256	HCP-00242	Dr. Kavita Sharma	5	T-040	2024-10-03	9999-12-31	true

The old row is *closed* — `valid_to` set to the day before the change, `is_current` false — and a new row is opened. Nothing is edited in place except those two closing columns.

What a plain UPDATE would break. Overwrite row 255's territory instead, and all 400 prescription rows Dr. Sharma wrote before the move now point at a row saying T-040. The T-035 rep's history rewrites itself, last year's league table changes every time somebody transfers, and no query can reconstruct what the numbers looked like when a decision was made.

Temporal key resolution. The other half of Type 2 is the lookup. `resolve_surrogate_keys` joins facts to `dim_hcp` on `hcp_id`, then keeps only the version whose validity window contains the fact's own date:

```
work = work.merge(hcp, on="hcp_id", how="left")
in_window = (work[date_col] >= work["valid_from"]) & (work[date_col] <= work["valid_to"])
work = work[in_window | work["hcp_key"].isna()]
```

A script dated March resolves to `hcp_key` 255 and picks up T-035; one dated November resolves to 256 and T-040. The split is visible in the loaded facts: 400 prescription rows on key 255 credited to T-035 (through 2024-10-02) and 644 on key 256 credited to T-040. A naive "join to the current version" load would have put all 1,044 in T-040.

The invariant, enforced by Postgres rather than by convention:

```
CREATE UNIQUE INDEX uq_hcp_current ON warehouse.dim_hcp (hcp_id) WHERE is_current;
```

A partial unique index is exactly right here: uniqueness must hold only over the current slice, because closed historical rows for the same `hcp_id` are free to repeat and a plain unique constraint would forbid them. This load produced 38 closed history rows across 2,000 HCPs.

5. Why staging is all TEXT

Every column in `staging.stg_hcp`, `stg_prescriptions` and `stg_sales_calls` is `TEXT`, with no constraints. Source extracts lie. A real feed will eventually hand you `'N/A'` in a numeric column, and if staging declares that column `INTEGER` the `COPY` aborts — one bad row out of two million kills the whole load, at the boundary where you have the least context to do anything about it.

Typing happens instead in `coerce_columns`, where a failure costs you that row and nothing else:

```
cast = pd.to_numeric(work[col], errors="coerce")
failed = cast.isna() & work[col].notna()
reasons[failed & ~bad] = f"non-numeric value in {col}"
```

A bad row takes one of two paths: it fails to cast, or it casts fine but its natural key resolves to no dimension row. Both end up in a rejects frame carrying a `reject_reason`, which the pipeline writes to `data/rejects/*.csv`. In this load:

Stage	Prescriptions
Staged into <code>staging.stg_prescriptions</code>	2,230,247
Loaded into <code>fact_prescriptions</code>	2,229,747
Rejected	500

The 500 split evenly: 250 rows with `"N/A"` in `trx_count` (uncastable numeric) and 250 with product code `UNKNOWN-99` (unresolvable natural key). `fact_sales_calls` loaded 68,619 rows with zero rejects.

The `UNKNOWN-99` half of that is load-bearing, and for a while it was false — not the count, the identities. `resolve_surrogate_keys` tracked its input rows by position and then selected the rejected ones back out of the original frame by index `label`, and `coerce_columns` had already punched 250 gaps in that index. Every label past the first gap picked the wrong row, so the quarantine file listed clean rows with a `reject_reason` bolted on and never named the ones that had actually failed. The warehouse itself was unaffected — the rows that failed were still kept out of the fact table — and every count reconciled exactly, which is precisely why nothing caught it for so long. Section 9 has the fix and the regression test that pins it.

The pipeline asserts **staged = loaded + rejected** on every run and raises `SystemExit` if it does not hold. That is the point of never silently dropping a row: a pipeline that cannot tell you where its rows went is not a pipeline.

6. The load path

`pandas.to_sql` issues parameterised `INSERTS`. Pandas batches them through `executemany`, so it is not literally two million round-trips, but it is still two million rows of statement parsing and parameter binding. `COPY ... FROM STDIN` streams the same data as one buffered CSV parsed server-side in a single pass, chunked at 250,000 rows so the serialised buffer never costs hundreds of megabytes of memory.

That was the easy part. The real bottleneck was not bytes, it was **foreign keys**. `fact_prescriptions` has four (date, HCP, product, territory) and `fact_sales_calls` four more — eight in total. With all eight enabled Postgres runs one index lookup per key per row: roughly 8.9 million checks for the prescription load alone. The first attempt at the fact `COPY` **never completed**; it was killed at **14 minutes 48 seconds**, still running.

The fix is `constraints_suspended`, a context manager that reads the constraint definitions out of `pg_constraint`, drops them, yields, and re-adds them verbatim in `finally`:

```
with ls.constraints_suspended(conn, fact_tables) as n_dropped:
    ls.copy_dataframe(conn, rx_resolved, "warehouse.fact_prescriptions", cols)
    ls.copy_dataframe(conn, calls_resolved, "warehouse.fact_sales_calls", cols)
# constraints re-added here
```

Fact load	Time
Foreign keys enabled	> 14m 48s (killed, never finished)

Fact load	Time
Foreign keys suspended, revalidated afterwards	41–66 s

Full pipeline end to end, cold run to warm run: **150–246 seconds**.

The guarantee is unchanged, not weakened. Re-adding a foreign key makes Postgres validate the whole table — one sequential scan and hash join per constraint instead of one index probe per row — and the fact COPY and that revalidation happen inside the *same transaction*: the pipeline runs with `autocommit = False` and commits only after the constraints are back. If any loaded row violated a key, the `ADD CONSTRAINT` fails, `conn.rollback()` runs, and every fact row goes with it. There is no window in which the committed database holds unvalidated facts. Staging and the four dimensions are committed before this point, so the run is not one transaction end to end — but the next run drops and recreates both schemas anyway. It is faster, not laxer.

7. The four analytics queries

`hcp_decile_ranking.sql` — who do we call on?

Which prescribers drive our brand inside each territory? Aggregates own-brand TRx per HCP, then applies `NTILE(10)` and `RANK()`, both `PARTITION BY territory_key ORDER BY total_trx DESC` — within territory, because a rep can only call on doctors in their own geography. Output columns are `hcp_id`, `full_name`, `specialty`, `territory_name`, `total_trx`, `calculated_decile` and `territory_rank`.

The grain is one row per HCP *version* per territory — 2,038 rows for 2,000 HCPs — because the query groups by `hcp_key`, the SCD2 surrogate, not by `hcp_id`. An HCP who moved mid-period is therefore ranked in each territory that earned the volume, which is the payoff of section 4 rather than a wrinkle to apologise for. Territory 001's top row is 2,535 TRx at rank 1.

`brand_share_mom.sql` — are we winning or just floating?

What share of the market does each brand hold each month, and which way is it moving? `SUM(trx) OVER (PARTITION BY year_month)` gives the monthly denominator; `LAG(share_pct) OVER (PARTITION BY brand_name ORDER BY year_month)` gives the change in percentage points. Competitor products are deliberately not filtered out — the denominator has to be the whole market or "share" means nothing. Unedited output for 2024-02: Pulmovent at 18.68% share, +0.51 pp on January, against Cardiova at 18.39%, -0.09 pp.

`territory_attainment.sql` — who is behind plan?

Which territories are missing quota, and how deep is the year-to-date hole? A running total via `SUM(...) OVER (PARTITION BY territory_key, year ORDER BY year_month ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)` — an explicit `ROWS` frame rather than the default `RANGE`, so the total is cumulative row by row and resets each January. Output is one row per territory-month: actual, target, gap, monthly and YTD attainment, rank, and a `plan_status` traffic light. There is no target table here, so quota is derived — each territory's first six months averaged, times 1.08 — and those months are labelled `BASELINE PERIOD` rather than passed off as performance.

`call_effectiveness.sql` — do calls work?

Do HCPs called on in the previous 14 days write more? Both facts are rolled up to HCP-day before the range join (matching 2.2M raw rows straight against the call fact is what makes the naive version unusable), then matched on `call_date BETWEEN rx_date - 14 AND rx_date - 1` — exclusive of the script day, because a call and a script on the same day is not evidence of anything. `GROUPING SETS` produces national and per-decile rows in one pass so totals cannot drift from detail, and a windowed `MAX(...)` `FILTER (WHERE cohort = 'NOT CALLED')` `OVER (PARTITION BY stratum)` carries the uncalled baseline onto the called row so the lift reads off one line. Output is one row per decile stratum per cohort, with HCP-months, average and median TRx, and the lift.

Called HCP-months average **39.31 TRx** against **23.71** uncalled, a **+65.8%** lift — largest in the *low* deciles (decile 01: +94.5%, decile 02: +73.7%).

Read that carefully. The generator deliberately plants a 14-day post-call lift, so this verifies that the pipeline and query recover a known signal. It is a correctness check on the system, not a claim about real medicine. Even with real data the comparison would be observational and confounded in the obvious direction — reps choose to call on high prescribers — which is why the query stratifies by decile.

8. Indexing: what helped and what did not

`sql/02_indexes.sql` is deliberately separate from the schema file, so neither fact table carries an index beyond its primary key and the un-indexed state is an honest baseline. `etl/measure.py` drops the indexes, measures, restores them, and measures again. Every figure is the **median of three runs after a discarded warm-up** — measuring "before" cold and "after" warm manufactures a speedup you did not earn.

Query	No index	Indexed	Change	Plan when indexed
Selective drill-down (WHERE hcp_key = 742)	146.9 ms	3.5 ms	42x faster	Bitmap Heap Scan / Bitmap Index Scan
Month-over-month brand share (full aggregate)	2181.1 ms	2139.6 ms	no change	Parallel Seq Scan

The first row is the expected win: with 2,000 HCPs, one prescriber's rows are about 0.05% of the table, so the plan flips from Parallel Seq Scan to a Bitmap Index Scan and the query gets 42 times faster.

The absolute milliseconds belong to one laptop and one run, and they move. Repeated invocations of `make tune` on this machine have put the un-indexed drill-down anywhere between 103 ms and the 146.9 ms above, and the indexed one between 3.1 ms and 3.7 ms — a ratio band of roughly 28x to 42x. What reproduces every time is the plan change and the order of magnitude, and those are the parts worth believing. Run it yourself rather than taking the table's word for it.

The second row is the interesting one. The composite index does nothing for the brand share report, and that is correct behaviour rather than a failure: the query aggregates every row in the table, so there is no selective predicate, nothing for an index to narrow, and a parallel sequential scan is genuinely the right plan. An earlier version of this benchmark appeared to show a 1.91x win there. It was entirely cold-versus-warm cache and it vanished under fair measurement. It is reported rather than quietly dropped, because the corrected number is the more useful one.

There was a third row here once, and deleting it is the same discipline. It claimed that a date-and-product-filtered variant got roughly 2.5x *slower* with the composite index in place, the planner having traded a parallel scan for a nested loop of per-day index lookups. It is gone because it would not reproduce, and because there was no exact SQL behind it to re-run: `etl/measure.py` measures two queries and neither of them is that one. The failure mode is real and worth knowing about — an index on a predicate that is not selective can tempt the planner into per-row lookups and cost it parallelism — but this repository has not demonstrated it, and an unreproducible figure in a document that opens by promising every number was measured is worse than no figure at all.

*An index earns its keep on **selectivity**, not on table size.*

The four indexes that ship do earn it. `idx_rx_hcp` is justified by the workload in the table above — the 42x drill-down — and `idx_rx_territory` by the per-territory rollups in `territory_attainment.sql`. Both also back a foreign key from `fact_prescriptions`, and it is worth being precise about what that is worth here, because it is easy to overclaim. Postgres indexes the *referenced* side of a foreign key automatically but never the *referencing* side, so `DELETE`ing a dimension row, or changing an `hcp_key`, has to find the children — a full scan of the fact table without an index on the referencing column. This pipeline does neither: it is full-refresh, `dim_hcp` is dropped and rebuilt by `COPY` every run, and SCD2 closes rows inside a pandas frame, so there is no `UPDATE` statement anywhere in `etl/` or `sql/`. Even if there were, a referential-integrity check fires only when the *referenced key columns* change, and closing a row touches `valid_to` and `is_current`, not `hcp_key`. The foreign-key support is insurance for a future incremental loader; the query workload is what pays for these two indexes today.

`idx_calls_hcp_date` is ordered (`hcp_key`, `date_key`) because the equality column must lead; reversed, the date range becomes the leading bound and `hcp_key` degrades into a filter over the whole window. `idx_rx_date_product` carries `INCLUDE (trx_count)` so a date-ranged aggregate can be answered index-only — useful for period-bounded reports, explicitly not for the unbounded one measured above. The file ends with an `ANALYZE`: an index the planner has no statistics for is an index it will not choose.

9. Testing

36 tests, in two halves that do genuinely different jobs.

`tests/test_transform.py` — **15 pure unit tests, no database.** SCD2 closes the old row and opens exactly one new current one on a tracked change; an untracked change (a rename) is a no-op, as is an identical reload; unresolvable keys go to rejects with the right reason; and `loaded + rejected == input`, always. The temporal test asserts that a March fact and an August fact for one HCP resolve to different surrogate keys *and* different territories.

One of them, `test_rejects_survive_a_gapped_index`, exists because of a bug that got through everything else. `resolve_surrogate_keys` tracks input rows by position but originally selected the rejected ones back out with `.loc`, which is label-based — and by the time the pipeline calls it, `coerce_columns` has already punched gaps in the index where it dropped type-rejects. Every label past the first gap resolved to the wrong row, so the quarantine file named clean rows and never named the bad ones. The fact table was fine — the rejected rows were excluded from it correctly, by a mask that was never in doubt — and the counts reconciled perfectly, which is exactly why nothing caught it: every assertion in the suite was counting rows, and the counts were right. The regression test asserts on *identity*, that the row in the reject file is the row that actually failed, because that is the only thing that fails when this breaks. The fix is one character: `.iloc`, not `.loc`.

They run in milliseconds against a handful of hand-written rows, with no Postgres anywhere near them, which is only possible because `etl/transform.py` contains no I/O at all. Writing the transform as pure functions was a testing decision first and a tidiness decision second.

`tests/test_data_quality.py` — **21 assertions against the loaded warehouse.** The checks you would run after every production load: row-count reconciliation against the reject files, zero orphan foreign keys across all eight fact-to-dimension relationships, no nulls in the measures, `nrx_count <= trx_count`, the SCD2 invariants (one current row per HCP, history exists, no two versions claim the same day), and — the important one — that every fact resolves to the temporally correct HCP version:

```
SELECT count(*) FROM warehouse.fact_prescriptions f
JOIN warehouse.dim_hcp h USING (hcp_key)
JOIN warehouse.dim_date d USING (date_key)
WHERE d.full_date < h.valid_from OR d.full_date > h.valid_to;
```

That must be zero; a load that resolved keys to "whichever version is live now" would fail it on every historical row. The DB fixtures **skip** rather than fail when Postgres is unreachable or the warehouse is empty, so the unit half stays runnable anywhere.

10. Limitations, and what changes at 200M rows

Honest gaps in what exists today:

- **The pipeline is full-refresh.** `apply_scd2` supports incremental folding — the unit

tests exercise exactly that — but `pipeline.py` calls it with an empty existing state and rebuilds the dimension every run. No watermark, no upsert, no restartability.

- **There is no target fact table.** Territory quota is derived arithmetically inside

`territory_attainment.sql`: documented as such, but not a real feed.

- **Two decile conventions coexist.** `dim_hcp.decile` runs 10 = highest volume (confirmed

against the load: decile 10 averages 2,568 own-brand TRx, decile 1 averages 45), while `NTILE(10) ... ORDER BY total_trx DESC` in `hcp_decile_ranking.sql` makes 1 = top. Comparing the two to find stale segmentation, as that query's header suggests, currently compares inverted scales.

At 200 million rows the shape of the problem changes:

- **Partition** `fact_prescriptions` **by month**. At 2.2M rows a parallel sequential scan is

fine; at 200M the brand-share aggregate needs partition pruning — the real fix in precisely the place where an index measurably was not.

- **Key resolution moves out of pandas**. The fact path already streams — staging is read in

250,000-row batches through a server-side cursor, so peak memory is bounded (442 MB measured for the full load) and set by the batch size rather than the table. But each batch still merges against the dimensions in pandas, and at two hundred million rows the temporal join belongs in the database as an `INSERT ... SELECT` range-joined to `dim_hcp`.

- **Incremental loads become mandatory** — a watermark on `date_key` plus an upsert, and a backfill path for late-arriving facts, which does not exist today.

- **Constraint suspension stops being enough**. Per-partition loads, `ATTACH PARTITION` after validation, and pre-aggregated monthly rollups so dashboards never touch the base fact.

- **Orchestration**. Airflow or dbt for scheduling and lineage, once there is more than one pipeline and someone other than the author needs to know what ran when.